

实验一：

1. 程序说明：编写 Python 程序实现功能：从键盘输入若干姓名，保存在字符串列表中；输入任意姓名，检索列表中是否存在。

2.

2. 程序实现：

```
# -*- coding: utf-8 -*-
"""
```

Created on Thu Sep 14 09:11:10 2023

```
@author: 86157
"""
```

```
l = [];
for i in range(5):
    print("输出第", i+1, "个学生姓名:");
    s = input();
    l.append(s);

#%%
print(l);
n = input("请输入一个学生的姓名: ");
if n in l:
    print("yes");
else:
    print("no");
```

3. 运行结果：

```
In [4]: runfile('D:/桌面/python/1-1.py', wdir='D:/桌面/python')
输出第 1 个学生姓名：
```

```
任超越
输出第 2 个学生姓名：
```

```
马胜涛
输出第 3 个学生姓名：
```

```
王文昊
输出第 4 个学生姓名：
```

```
王梓庚
输出第 5 个学生姓名：
```

```
刘东让
['任超越', '马胜涛', '王文昊', '王梓庚', '刘东让']
```

```
请输入一个学生的姓名：王三
no
```

```
In [5]:
```

实验二：

1.程序说明：

“大润发”、“沃尔玛”、“好德”和“农工商”四个超市都卖苹果、梨、香蕉、桔子和芒果五种水果。使用 NumPy 的 ndarray 实现以下功能。

- (1) 创建 2 个一维数组分别存储超市名称和水果名称；
- (2) 创建 1 个 4×5 的二维数组存储不同超市的水果价格，其中价格由 4 到 10 范围内的随机数生成；
- (3) 选择“大润发”的苹果和“好德”的香蕉，并将价格增加 1 元；
- (4) “农工商”水果大减价，所有水果价格减少 2 元；
- (5) 统计四个超市苹果和芒果的销售均价；
- (6) 找出桔子价格最贵的超市名称（不是编号）。

2.程序实现：

```
# -*- coding: utf-8 -*-  
"""
```

Created on Sat Sep 16 14:54:49 2023

```
@author: 86157  
"""
```

```
#导入 numpy  
import numpy as np
```

```
names = np.array(['大润发', '沃尔玛', '好德', '农工商'])  
fruit = np.array(['苹果', '梨', '香蕉', '桔子', '芒果'])  
###
```

```
#创建二维数组  
price=np.random.randint(4,11,size=(4,5))
```

```
###  
#选择“大润发”的苹果和“好德”的香蕉，并将价格增加 1 元；  
print(price[names=="大润发",fruit=='苹果']+1)  
print(price[names=="好德",fruit=='香蕉']+1)
```

```
###  
#使用 subtract 函数为农工商水果减去 2 元  
print( np.subtract(price[names=='农工商'], 2) )  
###统计四个超市苹果和芒果的销售均价；  
print(price[:,(fruit=='苹果')|(fruit=='芒果')].mean(axis=0))  
#print(price.sum(axis=0)/5)  
#print()
```

```
###
```

#找出桔子价格最贵的超市名称（不是编号）。

```
d=print(names[price[:,fruit=='桔子'].argmax()])
```

```
print(names[d])
```

3.运行结果:

	0	1	2	3	4
0	5	4	10	4	5
1	7	7	8	9	8
2	5	4	5	8	10
3	5	4	7	5	8

随机生成的二维数组表示各个超市水果的价格

```
In [5]: runfile('D:/桌面/python/1-2.py', wdir='D:/桌面/python')
```

```
[6]
```

```
[6]
```

```
[[3 2 5 3 6]]
```

```
[5.5 7.75]
```

```
沃尔玛
```

```
[['大润发' '沃尔玛' '好德' '农工商']]
```

实验三:

1.程序说明:

根据银行储户的基本信息，完成以下分析。

- 1) 从“bankpep.csv”文件中读取用户信息。
- 2) 查看储户的总数，以及居住在不同区域的储户数。
- 3) 计算不同性别储户收入的均值和方差。
- 4) 统计接受新业务的储户中各类性别、区域的人数。
- 5) 将存款账户、接受新业务的值转化为数值型。
- 6) 分析收入、存款账户与接收新业务之间的关系。

2.程序实现:

```
import os #调取 operating system 模块
```

```
cwd = os.getcwd()
```

```
print(cwd)
```

```
###
```

```
import pandas as pd
```

```
import numpy as np
```

```
from pandas import Series, DataFrame
```

```
#从 D:/桌面/python/bankpep.csv 文件读出数据
```

```
bankpep_data = pd.read_csv('D:/桌面/python/bankpep.csv')
```

```

#方法一
#print(bankpep_data['id'].count())

#方法二
#print(bankpep_data.count())

#方法三
#print(bankpep_data.info())

#方法四
#print(bankpep_data.index)

###
#求不同区域的储户数
print(bankpep_data.groupby('region')['id'].count())

###
#计算不同性别储户收入的均值和方差。
print(bankpep_data.groupby(['sex']).aggregate({'income':['mean','var']}))

###
#统计接受新业务的储户中各类性别、区域的人数。
#方法一
print(bankpep_data.groupby(['sex','region'])['pep'].count())
#方法二
print(bankpep_data[['sex','region','pep']].groupby(['sex','region']).count())

###
#将存款账户、接受新业务的值转化为数值型。
bankpep_data[['save_act','pep']]
=np.where(bankpep_data[['save_act','pep']]=='YES','1','0')
print(bankpep_data)

###
#分析收入、存款账户与接收新业务之间的关系。
print(bankpep_data[['income','save_act','pep']].corr().round(1))#保留一位小数

```

3.运行结果:

查看储户的总数，以及居住在不同区域的储户数。

```

600
region
INNER_CITY    269
RURAL         96
SUBURBAN      62
TOWN          173
Name: id, dtype: int64

```

计算不同性别储户收入的均值和方差。

```
income
mean      var
sex
FEMALE  27831.368233  1.698137e+08
MALE    27216.694200  1.633458e+08
```

统计接受新业务的储户中各类性别、区域的人数。

```
sex      region
FEMALE   INNER_CITY    131
          RURAL         47
          SUBURBAN      30
          TOWN          92
MALE     INNER_CITY    138
          RURAL         49
          SUBURBAN      32
          TOWN          81
Name: pep, dtype: int64
```

将存款账户、接受新业务的值转化为数值型。

```
id  age  sex  region  ...  save_act  current_act  mortgage  pep
0   ID12101  48  FEMALE  INNER_CITY  ...    0         NO         NO    1
1   ID12102  40   MALE   TOWN      ...    0         YES        YES    0
2   ID12103  51  FEMALE  INNER_CITY  ...    1         YES        NO    0
3   ID12104  23  FEMALE   TOWN      ...    0         YES        NO    0
4   ID12105  57  FEMALE   RURAL     ...    1         NO         NO    0
..   ...   ...   ...   ...      ...    ...         ...    ...
595 ID12696  61  FEMALE  INNER_CITY  ...    1         YES        YES    0
596 ID12697  30  FEMALE  INNER_CITY  ...    1         YES        NO    0
597 ID12698  31  FEMALE   TOWN      ...    1         NO         NO    1
598 ID12699  29   MALE  INNER_CITY  ...    1         NO         YES    0
599 ID12700  38   MALE   TOWN      ...    0         YES        YES    1

[600 rows x 12 columns]
```

分析收入、存款账户与接收新业务之间的关系。

```
income
income    1.0
```

实验四：

1.程序说明：

文件 bankpep.csv 存放着银行储户的基本信息，请通过绘图对这些客户数据进行探索性分析。

- 1) 客户年龄分布的直方图和密度图
- 2) 客户年龄和收入关系的散点图
- 3) 绘制散点图观察账户（年龄，收入，孩子数）之间的关系，对角线显示直方图
- 4) 按区域展示平均收入的柱状图，并显示标准差
- 5) 多子图绘制：账户中性别占比饼图，有车的性别占比饼图，按孩子数的账户占比饼图
- 6) 各性别收入的箱须图

2.程序实现：

```
# -*- coding: utf-8 -*-
```

```
"""
```

```
Created on Thu Sep 21 08:46:01 2023
```

```
@author: 86157
```

```
"""
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
data = pd.read_csv('D:/桌面/python/bankpep.csv')
```

```
#客户年龄分布的直方图和密度图
```

```
data['age'].plot(kind = 'hist',bins = 10,density = True,title = 'Customer Age')
```

```
data['age'].plot(kind = 'kde',style = 'k-')
```

```
plt.xlabel('Age')
```

```
plt.ylabel('Density')
```

```
plt.show()
```

```
%%%
```

```
#客户年龄和收入关系的散点图
```

```
#方法一
```

```
data[['age','income']].plot(kind = 'scatter',x = 'age',y = 'income',marker = 's',s = 8,title  
= 'Customer Income',label = '(age,income)',grid = True,xlim = [0,80])    #s 设置点  
大小
```

```
plt.xlabel('Age')
```

```
plt.ylabel('Income')
```

```
plt.show()
```

```
%%%
```

```
#绘制散点图观察账户（年龄，收入，孩子数）之间的关系，对角线显示直方图  
pd.plotting.scatter_matrix(data[['age','income','children']],c = 'm')    #c 用来设置颜
```

色: m 为红紫色

```
plt.show()
```

```
###
```

```
#按区域展示平均收入的柱状图，并显示标准差
```

```
#方法一
```

```
import numpy as np
```

```
mean = data.groupby(['region']).agg({'income':np.mean})
```

```
std = data.groupby(['region']).agg({'income':np.std})
```

```
mean.plot(kind = 'bar',yerr = std ,rot = 45,title = 'Customer Income',legend = False,color = 'r')
```

```
plt.xlabel('Region')
```

```
plt.show()
```

```
###
```

```
#多子图绘制：账户中性别占比饼图，有车的性别占比饼图，按孩子数的账户占比饼图
```

```
sex_data = data.groupby(['sex'])['sex'].count()
```

```
car_data = data[data['car'] == 'YES'].groupby(['sex'])['sex'].count()
```

```
children_data = data.groupby(['children'])['children'].count()
```

```
fig = plt.figure(figsize = (7,6))
```

```
fig.add_subplot(2,2,1)
```

```
sex_data.plot(kind = 'pie',title = 'Customer Sex',startangle = 60,autopct = '%1.1f%%')
```

```
fig.add_subplot(2,2,2)
```

```
car_data.plot(kind = 'pie',title = 'Customer Car Sex',startangle = 60,autopct = '%1.1f%%')
```

```
fig.add_subplot(2,2,3)
```

```
children_data.plot(kind = 'pie',title = 'Customer Children',startangle = 60,autopct = '%1.1f%%')
```

```
plt.savefig('饼图.jpg',dpi = 400,bbox_inches = 'tight')
```

```
plt.show()
```

```
###
```

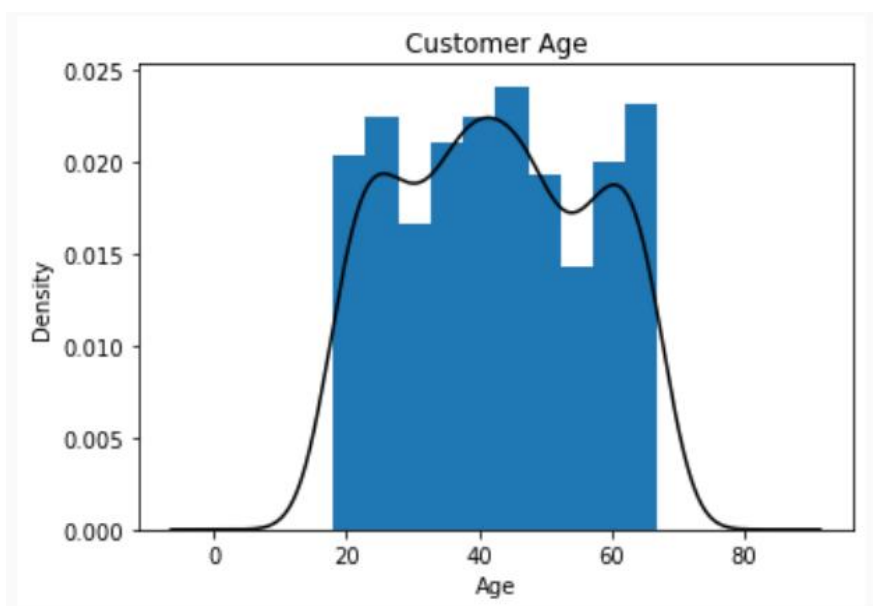
```
#各性别收入的箱须图
```

```
data[['income','sex']].boxplot(by = 'sex',figsize = (6,6))
```

```
plt.show()
```

3.运行结果:

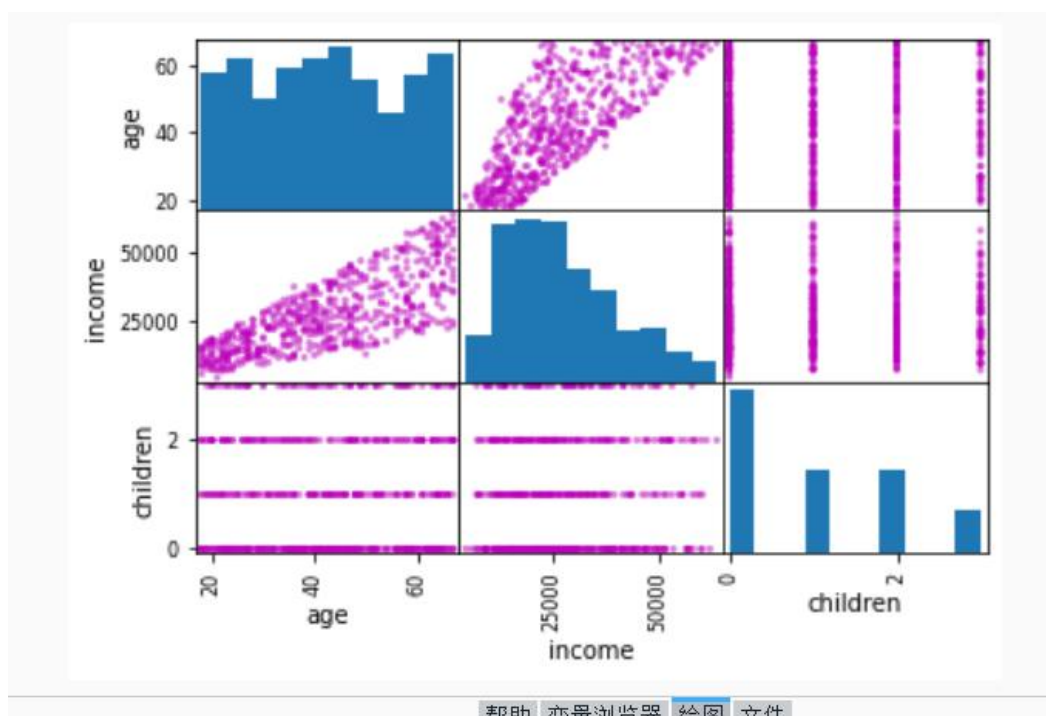
客户年龄分布的直方图和密度图



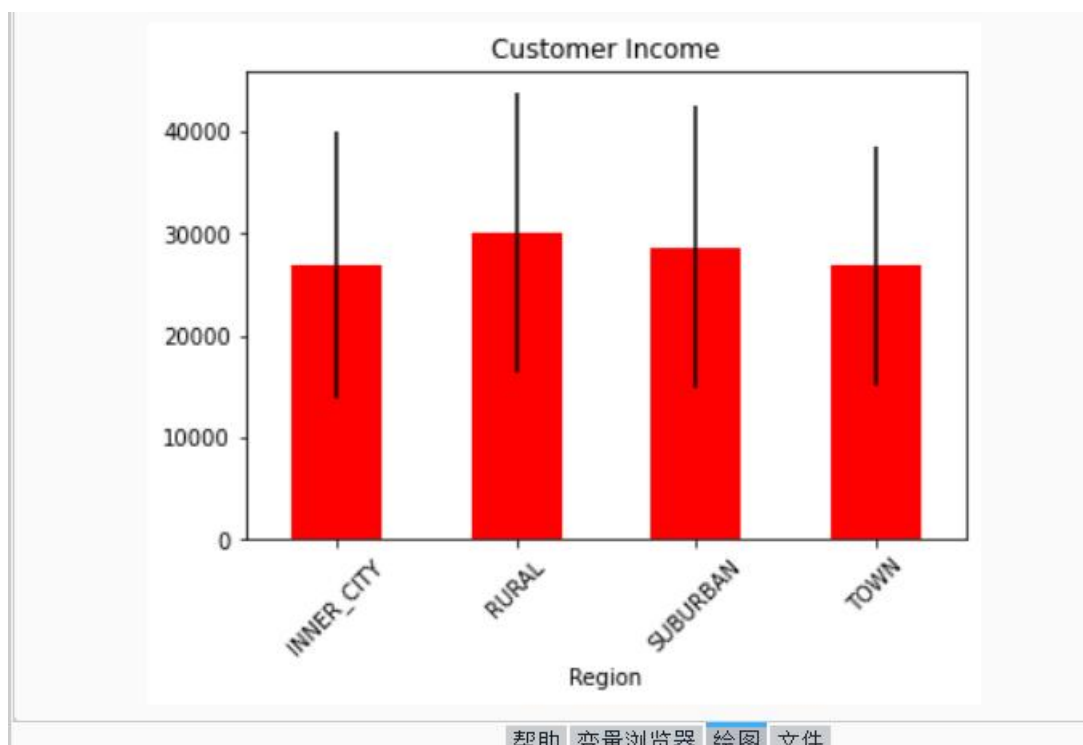
客户年龄和收入关系的散点图:



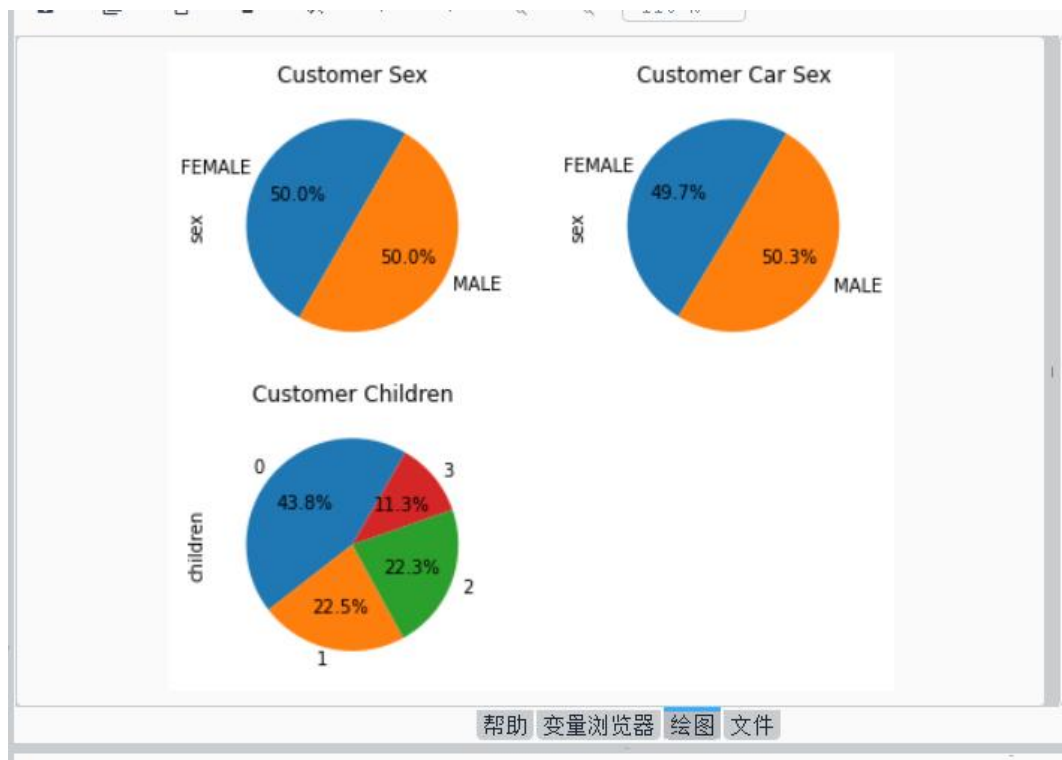
绘制散点图观察账户（年龄，收入，孩子数）之间的关系，对角线显示直方图



按区域展示平均收入的柱状图，并显示标准差



多子图绘制：账户中性别占比饼图，有车的性别占比饼图，按孩子数的账户占比饼图



各性别收入的箱须图：

